Transactions in EF Core

Module 10 • Lesson Summary

1. What Is a Transaction?

A **Transaction** groups multiple database operations into one logical unit. All operations should succeed together; if one fails, the transaction can be rolled back. This prevents partial updates and protects data consistency.

Common use cases: money transfers, placing orders, inventory updates, student enrollment, payment processing, and booking systems.

2. Banking Example

Transferring $100 from Mohammed to Ali requires two database updates:

Step	Operation
1	Withdraw $100 from Mohammed's account.
2	Deposit $100 into Ali's account.

Without a transaction: the withdrawal may succeed, then the application could crash before the deposit. Mohammed loses $100 while Ali receives nothing. This creates **Data Inconsistency**.

3. Transaction Workflow

Stage	Purpose
Begin Transaction	Start the transaction.
Execute operations	Withdraw from the sender and deposit to the receiver.
Commit	If all operations succeed, confirm the changes.
Rollback	If an operation fails, undo the transaction's uncommitted changes.

4. EF Core Example

```
using var transaction = context.Database.BeginTransaction();

try
{
// Update both tracked accounts
mohammed.Balance -= 100;
ali.Balance += 100;

context.SaveChanges();
transaction.Commit();
}
catch
{
transaction.Rollback();
throw;
}
```

BeginTransaction() starts an explicit transaction. **SaveChanges()** sends pending EF Core changes to the database. **Commit()** confirms the transaction, while **Rollback()** cancels its uncommitted changes.

5. SaveChanges() vs Commit()

SaveChanges() and **Commit()** are not interchangeable. SaveChanges persists pending EF Core changes within the current transaction. Commit completes the explicit transaction. A successful SaveChanges does not mean that an explicitly opened transaction has been committed.

6. Key Terms

Term	Meaning
Transaction	A group of operations treated as one logical unit.
Commit	Confirm the transaction's changes.
Rollback	Undo uncommitted changes in the transaction.
Data Inconsistency	Data left in an invalid or conflicting state.
Atomicity	The all-or-nothing property of a transaction.

Practical note: Real money-transfer logic must also validate sufficient funds, handle exceptions properly, and protect sensitive information. This simplified example focuses on transaction behavior.